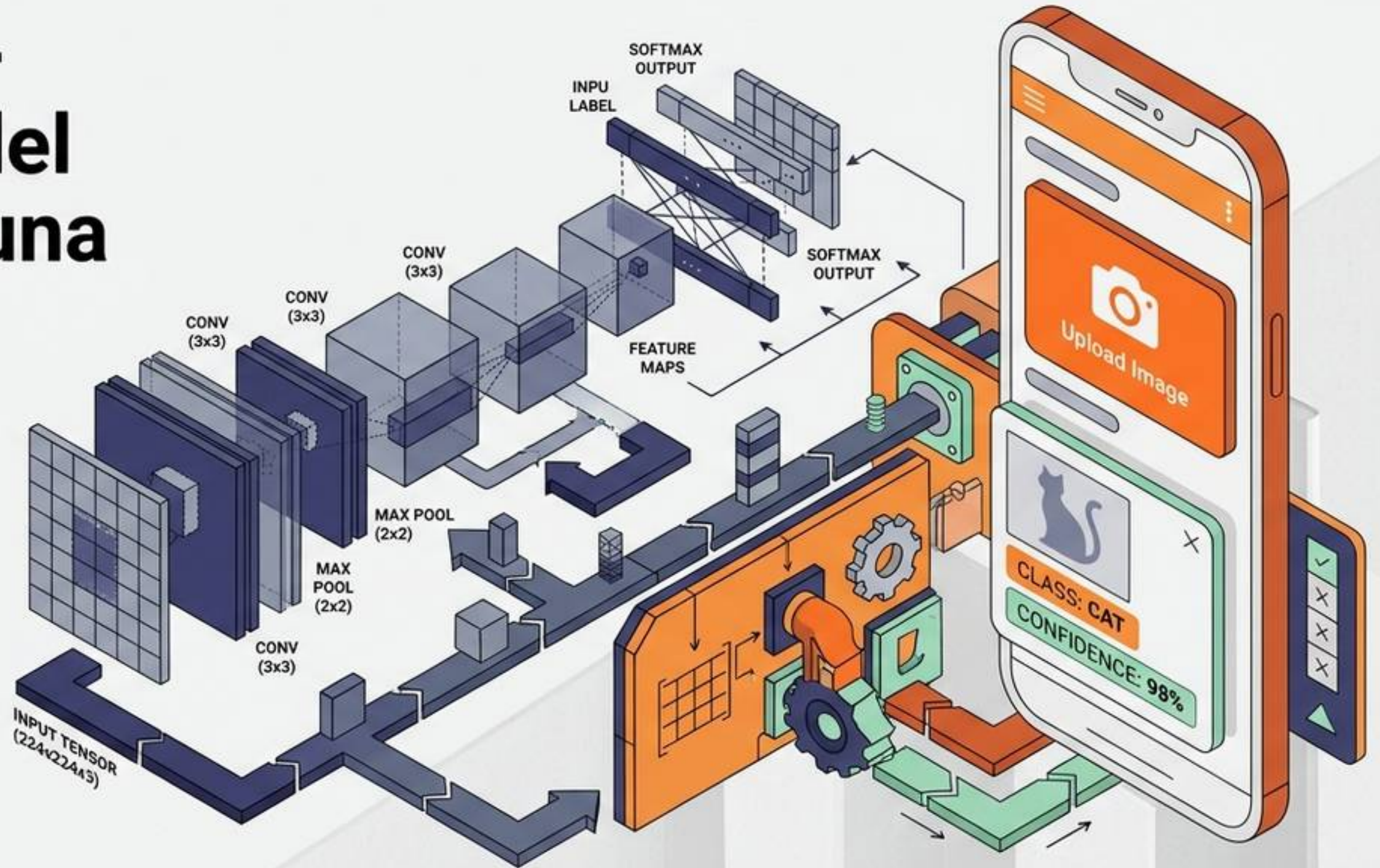
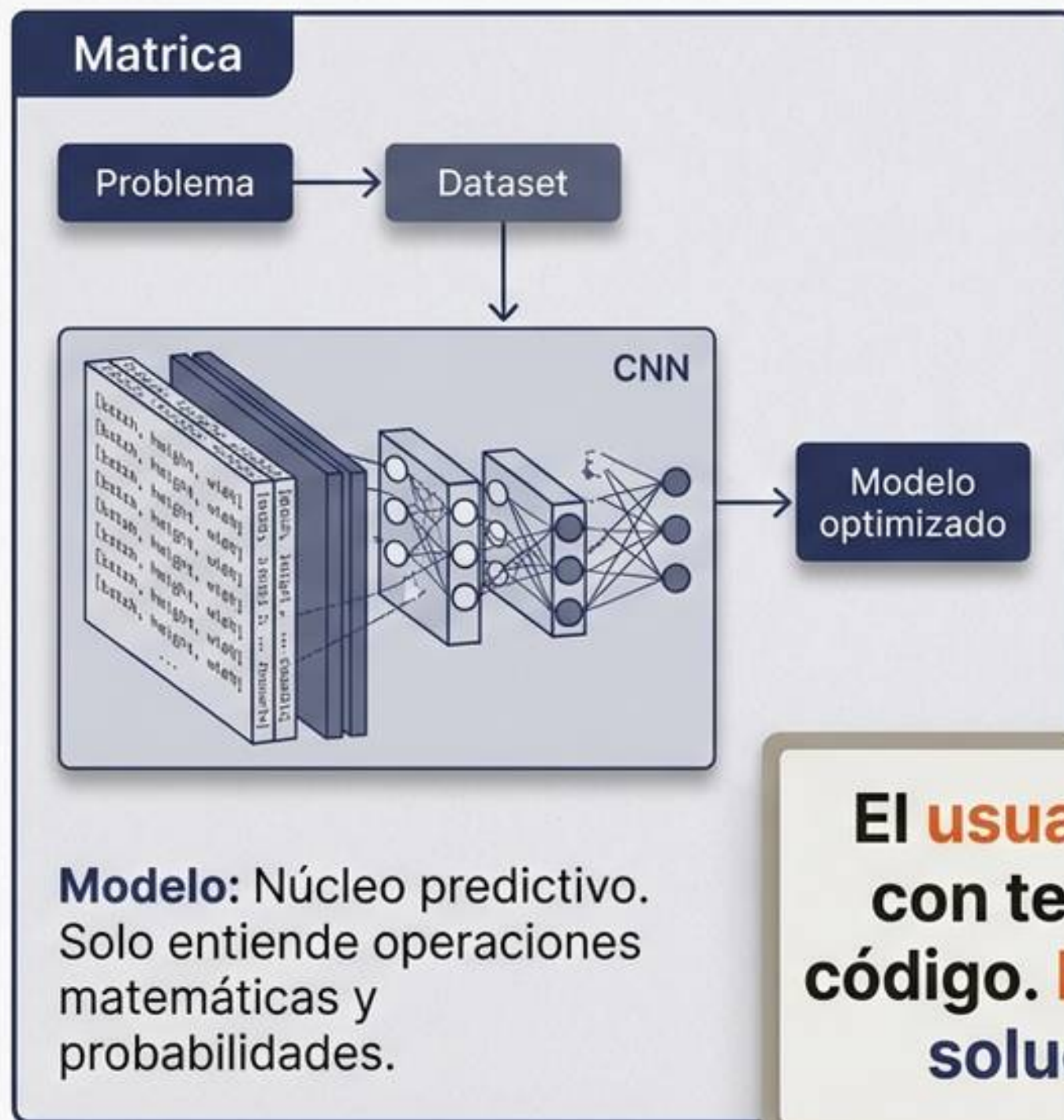


Semana 16 – Integración del Modelo con una Aplicación

Machine Learning 2026:
Del entorno de
experimentación al
producto funcional



Territorio del Modelo

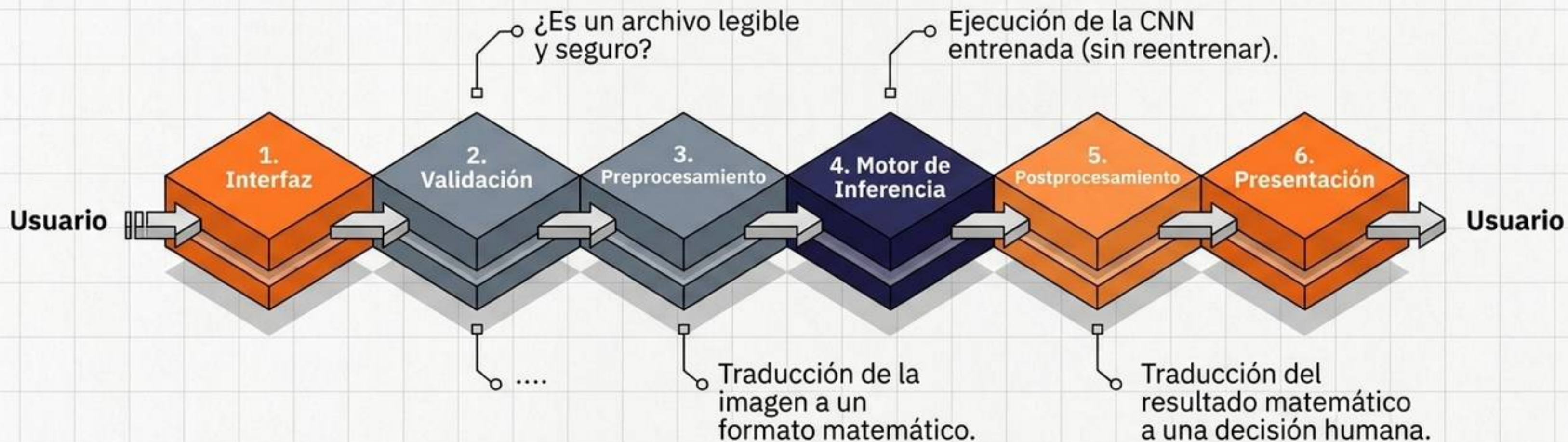


Territorio del Usuario

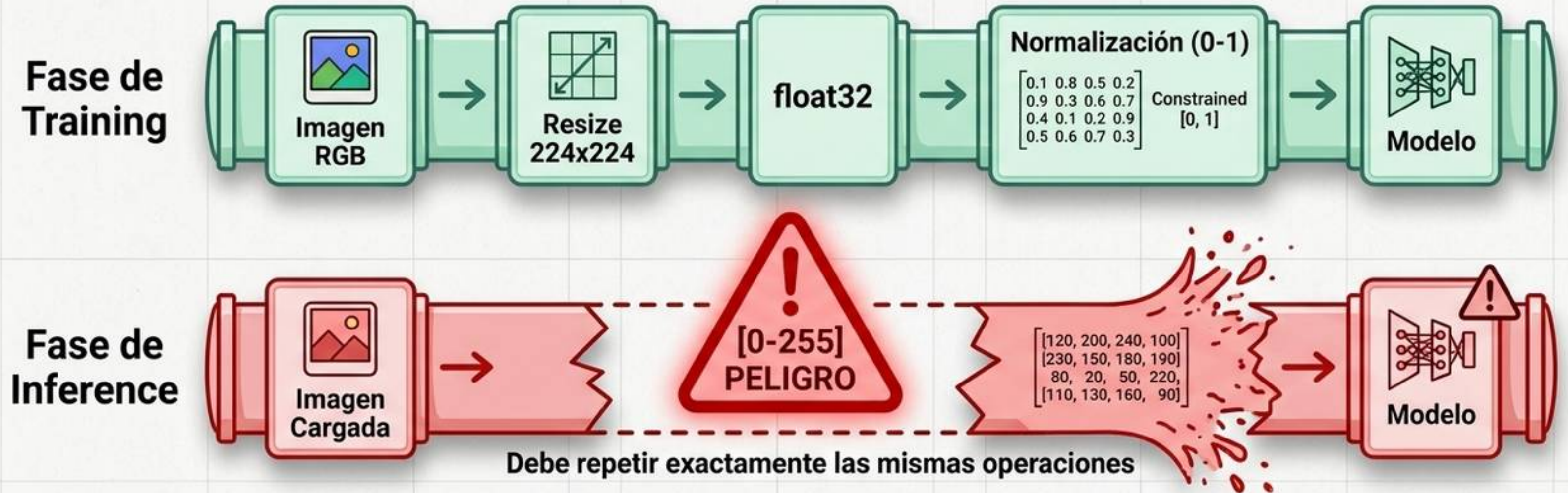


El **usuario** no interactúa con tensores, pesos ni código. **Interactúa** con una solución funcional.

Componentes de un Sistema de Inferencia



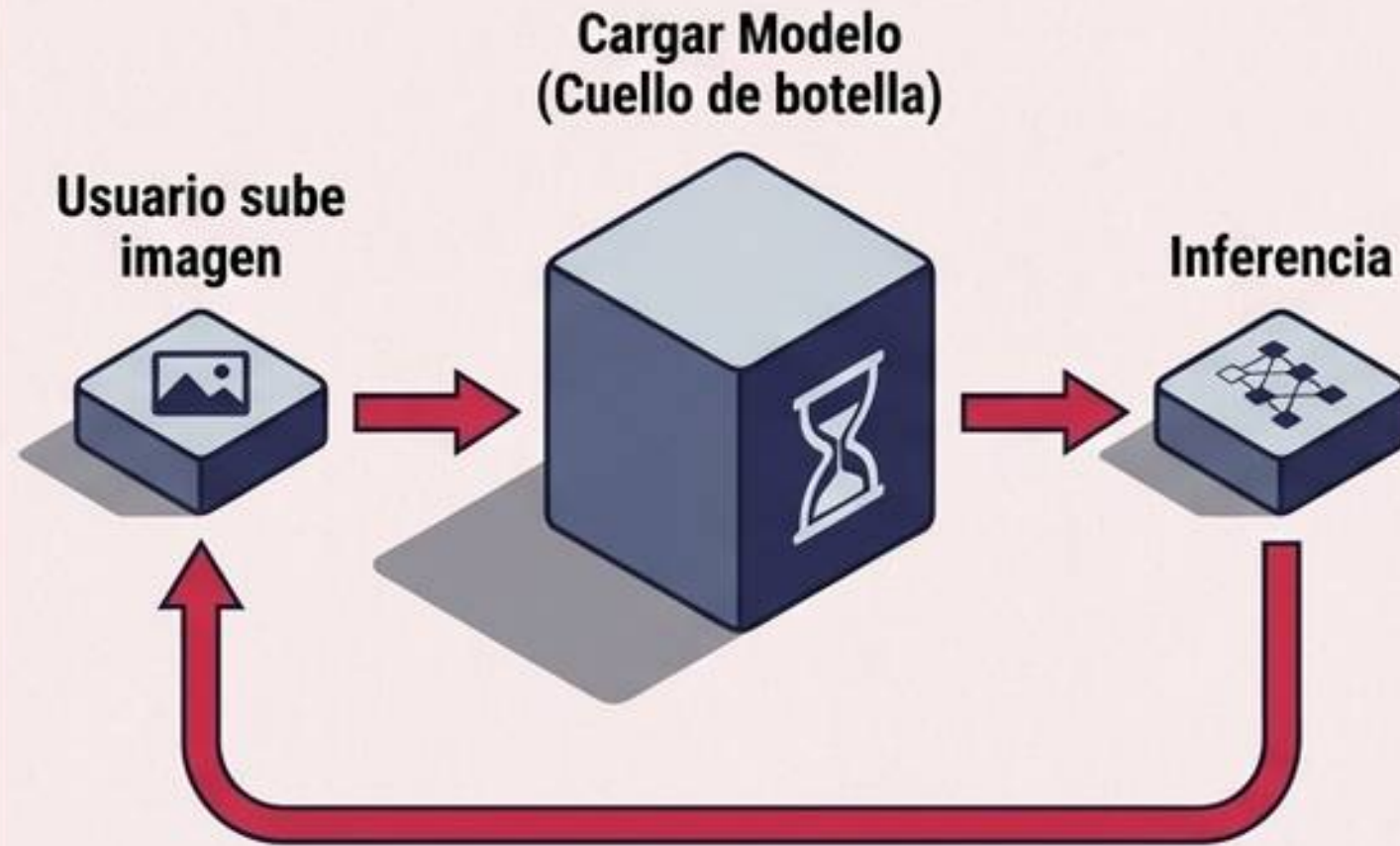
La Regla de Oro: Coherencia en el Preprocesamiento



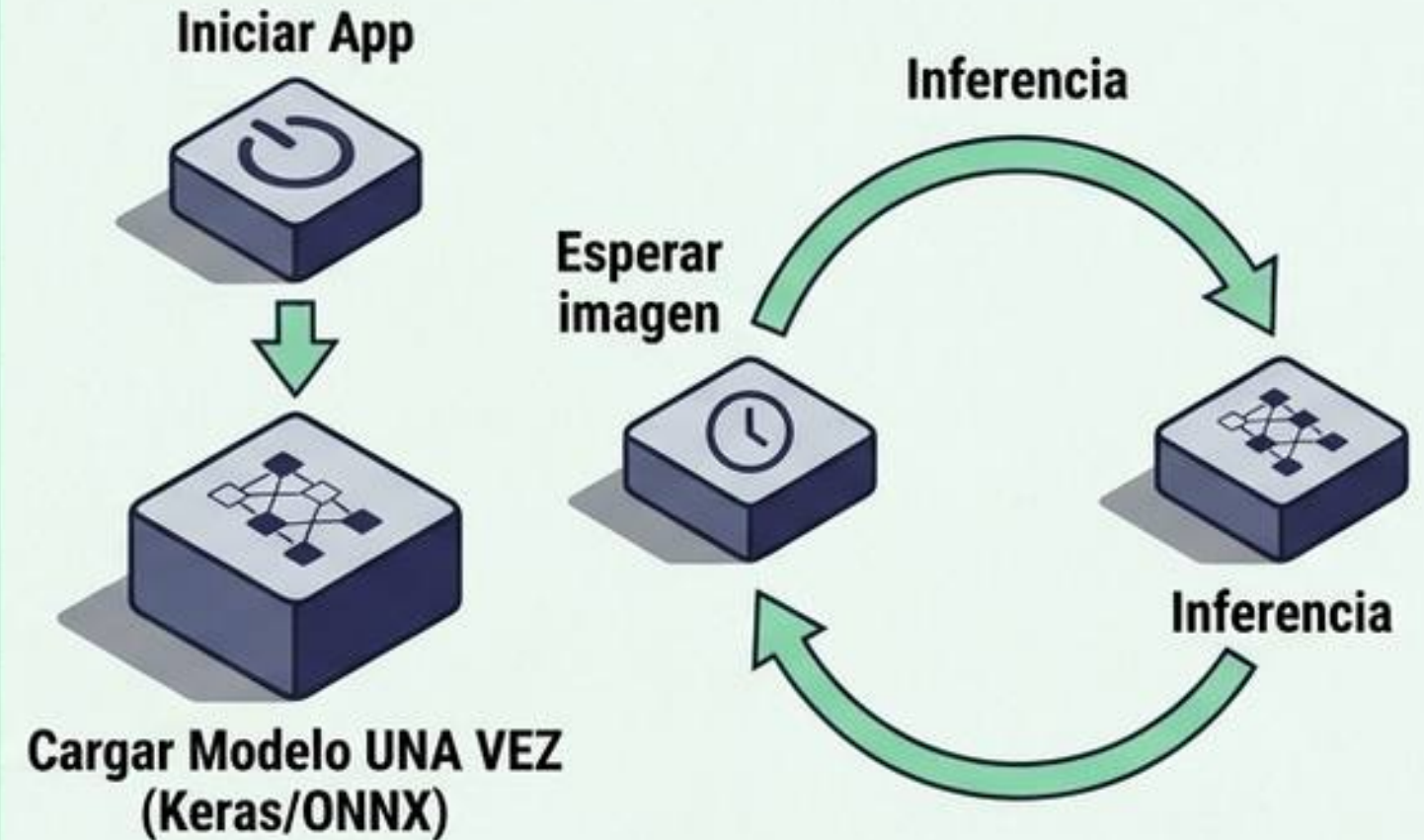
- El pipeline de inferencia **debe ser idéntico al de Training**.
- Si la distribución de entrada cambia (ej. valores 255 veces mayores), la CNN no arrojará error de código, pero la predicción será silenciosamente incorrecta.

Ejecución del Modelo: Eficiencia y Latencia

Mala Práctica



Buena Práctica



- **Inferencia Pura:** En esta etapa no existe entrenamiento, solo predicción directa.
- **El Costo de la Latencia:** Cargar un modelo de cientos de MB en cada ejecución destruye la experiencia del usuario.
- **Métrica Clave:** Es fundamental medir el Tiempo del Modelo vs Tiempo de Preprocesamiento en producción.

Del Vector Matemático al Significado Humano

1. Tensor de Salida crudo

[0.02, 0.13, 0.85, 0.00]

2. Operación de extracción

```
np.argmax(prediccion)
```

3. Índice Ganador

2

4. Contrato de Clases (Diccionario)

Índice	Clase
0:	'Vidrio'
1:	'Plástico'
2:	'Cartón'
3:	'Metal'

5. Resultado para el usuario

Categoría: **Cartón** 🛒

- La red neuronal solo devuelve un arreglo numérico crudo.
- El mapeo de índices a clases es responsabilidad del postprocesamiento de la aplicación.
- El orden de las clases es un contrato estricto entre el modelo y la aplicación.

Confianza $\neq$ Certeza (El Problema Out-of-Distribution)

Distribución Cerrada (Top-K)

Clase A (Cartón): 85%



Clase B (Papel): 10%

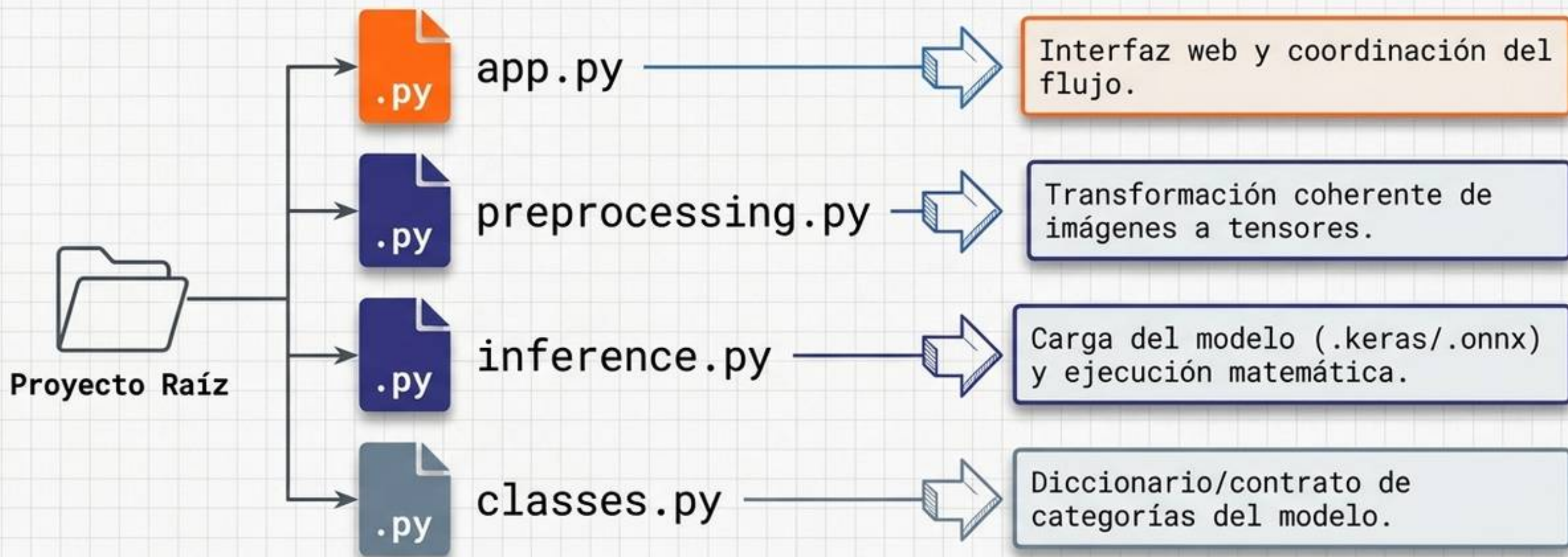
Clase C (Madera): 5%

Anomalía (Out-of-Distribution)



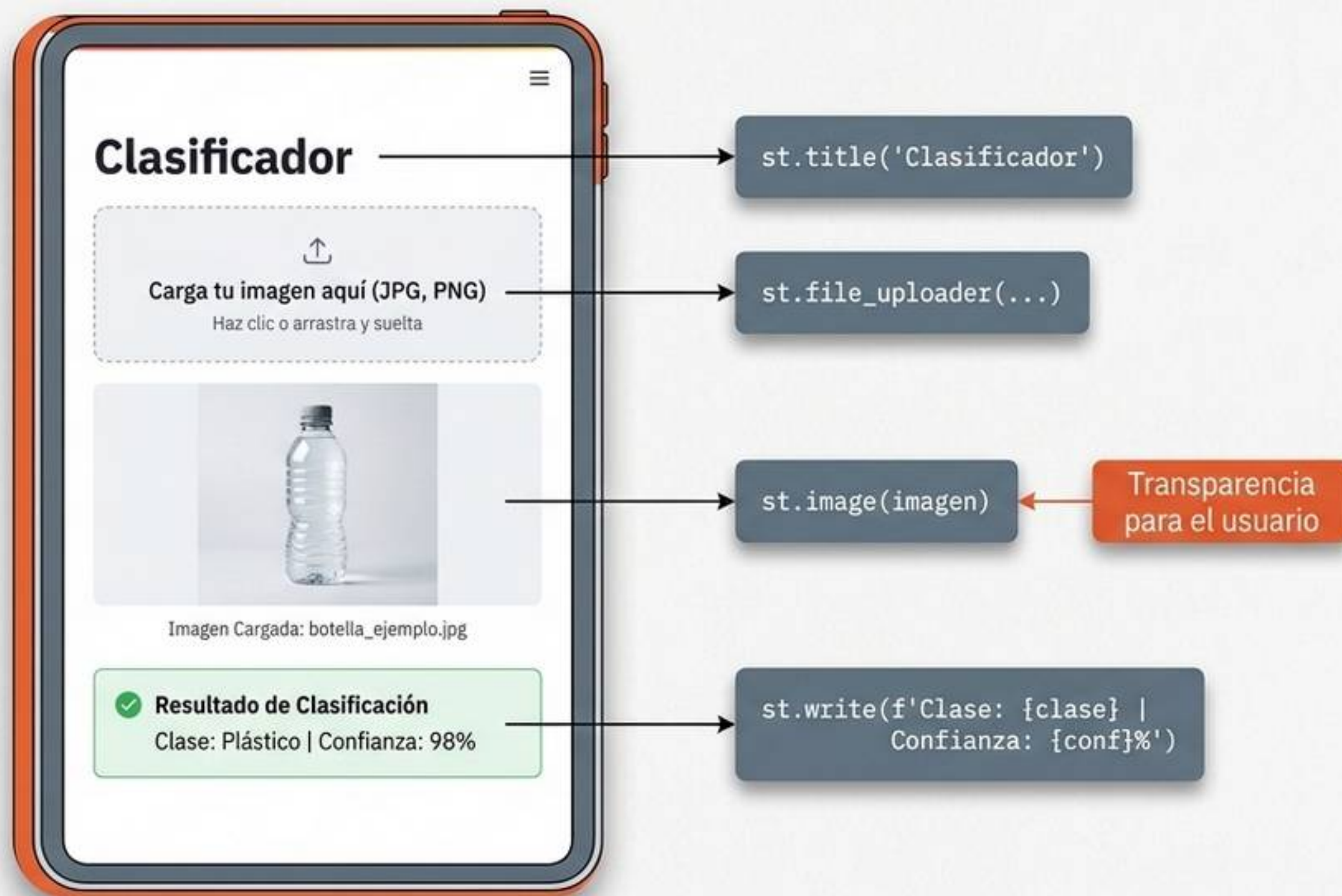
- **Confianza:** Porcentaje asignado a la clase dentro de su propia distribución cerrada, no una certeza absoluta sobre la realidad.
- **El Umbral:** Es necesario definir un porcentaje mínimo en la aplicación. Si no se supera, la UI debe mostrar 'Clasificación dudosa'.
- Los modelos cerrados siempre elegirán una categoría conocida, incluso ante entradas absurdas.

Arquitectura Modular del Proyecto



- **Separación de Responsabilidades:** Evita el código espagueti.
- **Escalabilidad:** Si el equipo decide cambiar la CNN propia por un modelo de Transfer Learning (ej. MobileNetV2), la interfaz gráfica (`app.py`) permanece intacta.

Interfaz Mínima Funcional con Streamlit



Streamlit

- Streamlit: Permite crear web apps en Python puro sin necesidad de programar HTML/JS.
- Transparencia: Mostrar la imagen cargada asegura al usuario qué datos está procesando el modelo.
- Objetivo: Solo se requiere cargar, previsualizar y clasificar para validar el Producto Integrador.

Estrategias de Despliegue y Privacidad

Aplicación Local (Streamlit + ONNX)



Privacidad:

- Alta (las imágenes no salen del equipo del usuario).

Red:

- Sin dependencia de Internet.

Gestión:

- Requiere instalación manual de dependencias.

Aplicación Cloud (Servidor / API)



Privacidad:

- Requiere políticas explícitas y gestión de eliminación de imágenes post-inferencia.

Red:

- Dependencia total, introduce latencia.

Gestión:

- Centralizada, acceso remoto universal.

Registrar predicciones es útil para MLOps, pero nunca almacenar imágenes de usuarios innecesariamente.

Identificando el Origen del Error



Error del Modelo

La aplicación funciona perfecto, pero clasifica sistemáticamente mal un patrón válido.



Error de Integración

La UI dice Metal pero el modelo internamente predijo Vidrio. (El postprocesamiento falló).



Error de Interfaz

El usuario presiona Clasificar y el frontend arroja un error de Python por un archivo no soportado.



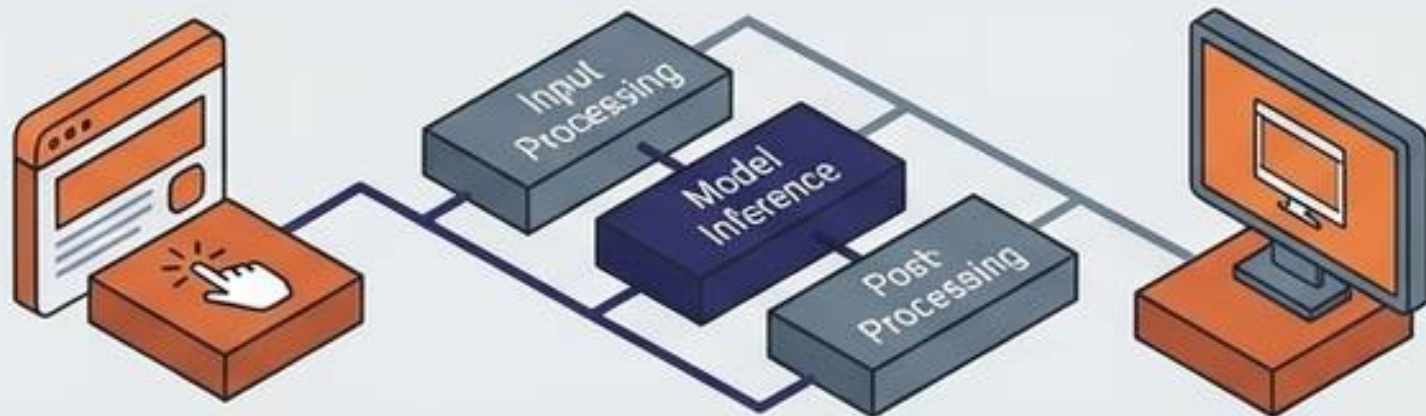
Error de Infraestructura

El modelo es demasiado pesado y agota la RAM del equipo, bloqueando la carga.

- Cuando el sistema falla, no siempre significa que la red neuronal está mal entrenada.
- Aislar la capa defectuosa es el primer paso del debugging.

Pruebas End-to-End y Validación Funcional

Validation Matrix		
Escenario de Prueba	Entrada	Resultado Esperado UI
Caso Ideal	JPG nítido de Clase A	Predicción correcta: Clase A ✓
Formato Inválido	Archivo PDF / Texto	Mensaje de error controlado ✗
Ambigüedad	Imagen muy borrosa	Advertencia Baja confianza ⚠
Mundo Real	Foto celular fuera del Dataset	Clasificación o rechazo robusto ✓



Prueba End-to-End: Valida todo el viaje: desde el clic del usuario hasta el píxel en pantalla.

Fuera del Dataset: Es imperativo probar la aplicación con imágenes nuevas (distinta luz, cámaras y fondos).

Síntesis de la Semana 16 (1/2)



1. Modelo $\neq$ Aplicación

El modelo es solo el cerebro; requiere una carrocería (UI) y tuberías de datos para interactuar con humanos.



2. Coherencia Absoluta

El preprocesamiento en producción (escalado, normalización) debe ser matemáticamente idéntico al del Training.



3. Traducción Crítica

La salida tensorial debe mapearse respetando estrictamente el contrato de clases establecido.



4. Confianza y Umbrales

Un modelo puede estar muy confiado de un error (Out-of-Distribution). Los umbrales evitan afirmaciones engañosas.



5. Carga Inteligente

El modelo se carga en memoria una sola vez al iniciar la aplicación, no en cada inferencia.

Diseño para el Mundo Real y Próximos Pasos



¿Dónde ejecutamos esta solución a escala?

Próxima estación: Despliegue Cloud, Edge Computing y MLOps.